

Конструирование программного обеспечения

Домашнее задание №1

SOLID · DI-контейнер · юнит-тесты

ВышКат

«Доедем до дедлайна»



Официальный (почти) брендбук сервиса «ВыиКат»: презентация уже есть, самокатов пока три, архитектура ещё в споре.

Преамбула

Национальный исследовательский университет «Высшая школа экономики» объявил о выходе на рынок кикшеринга. Да, конкуренты уже делят районы, инвесторы спрашивают экономику поездки, но у ВШЭ есть бренд, пилотный парк и уверенность, что **правильная доменная модель важнее яркого приложения.**

После трёх недель нейминга сервис называли «ВышКат» — потому что «Вышка» + «самокат», звучит бодро, а слоган «Доедем до дедлайна» уже нельзя убрать из пресс-релиза. Маркетинг торжествует. Разработка ещё не начиналась.

Ответственным за «техническую часть» по традиции назначили Факультет компьютерных наук. На ФКН сначала хотели доказать корректность проката формальными методами, затем набросать прототип на хакатоне «за выходные», а в промежутке успели поспорить, на каком стеке писать микросервисы для «трёх самокатов». В итоге договорились о компромиссе, достойном большой инженерии: консольное приложение, SOLID, DI-контейнер и юнит-тесты.

«Зато индустриальный подход», — сказал лектор по КПО и открыл эту домашку.

Заказчик проекта — ВШЭ, оператор сервиса «ВышКат». Подрядчиком по разработке по счастливой случайности оказались вы: **выход на рынок согласован, у вас есть две недели.**

Со слов заказчика: «Мы выходим на рынок всерьёз — «ВышКат» должен выглядеть как бизнес, а не как клубный прокат у корпуса. Техника попадает в активный парк только после техосмотра; нужно видеть суммарное энергопотребление флота и понимать, какие устройства можно безопасно выдавать новичкам. Инвентарные номера обязательны: без них это не кикишеринг, а квест «узнай чей это самокат». И пожалуйста, без ещё одного pet-проекта на коленке — ФКН же умеет делать нормальную архитектуру?»

Перед вами задача: разработать доменную модель и консольное приложение учёта парка «ВышКат» с соблюдением принципов SOLID и применением DI-контейнера. Ну и, разумеется, не забываем про тестирование!

Предметная область

Система в будущем будет развиваться: учёт транспорта, сотрудников службы эксплуатации, станций аренды и склада запчастей (можете продолжить фантазировать).

Но уже сейчас зафиксировано:

- на баланс ставятся транспорт и вещи (шлемы, док-станции, зарядные шкафы); сотрудники — нет;
- у устройств, потребляющих ресурс, хранится суточный расход энергии (кВт·ч);
- у устройств есть характеристика «простота для новичка» (1–10): при значении ≥ 6 устройство можно выдавать новичкам.

Ограничения и рекомендации по модели

Архитектор проекта озвучил ориентиры, не жёсткий шаблон. Иерархию и границы модулей проектируете вы и обосновываете в README.

- введите абстракции (интерфейсы) для всех потребителей энергии и для инвентаризируемых объектов (инвентарный номер);
- виды транспорта (самокат / велосипед / электровелосипед ...) с возможностью расширения без массовой правки кода;
- сервис техосмотра (ServiceCenter) решает, принимать ли новое устройство в парк;
- небольшая иерархия вещей для апробации (например, шлема, зарядные станции и др, что должно инвентаризироваться).

Требуется

1. Консольное приложение учёта парка «ВышКат», которое обеспечит: приём техники; отчёты по числу единиц и суммарной энергии; список прокатной техники, которая подходит для новичков; вывод наименования и инвентарных номеров всей техники и вещей.
2. README: логика решений, запуск, ответы на обязательные вопросы, роль ИИ (если был) и что переработано самостоятельно.
3. На проекте применить принципы SOLID и DI, но не переигрывайте, так как за «мёртвый» SOLID и overengineering вас уволят (снизим оценку). Каждый принцип обоснуйте в README.
4. Юнит-тесты на поведение, включая подмену техосмотра через DI (mock/fake/stub).

Функциональные требования

1. Добавление нового транспортного средства в парк.
2. Перед приёмкой — техосмотр (принять / отклонить).
3. Суммарное суточное потребление энергии парка.
4. Список устройств для новичков (база: «простота» ≥ 6 ; можете усложнить правило).
5. Учёт вещей на балансе и вывод инвентарных номеров вместе с техникой.
6. В консольном приложении продемонстрируйте все возможные сценарии приложения.

Обязательные вопросы в README

Коротко и по существу, со ссылкой на типы/модули кода:

- какой принцип SOLID применён наиболее явно и где;
- какой принцип сознательно ограничен / «почти нарушен» и почему это приемлемо;
- что сломается (или не сломается) при добавлении нового вида транспорта, сотрудника или склада запчастей (можете придумать свой вариант расширения программы);
- какой тест фиксирует решение техосмотра и/или правило «для новичков»; как подменяется зависимость;
- что сделал ИИ, а что изменили вы (если ИИ не использовался — укажите это).

Политика использования ИИ

ИИ-помощники разрешены как инструмент. Студент отвечает за понимание и качество кода. Если вы претендуете на 9 и 10 баллов, то вам нужно подтвердить свои знания на защите*: потребуется объяснить решение и «доработать» его под новое требование, не пояснённый код по соответствующим критериям считается невыполненным. Вы можете отказаться от защиты, но тогда ваши оценки (9 и 10) снижаются до 8 баллов.

* короткая онлайн защита (5–10 мин)

Критерии оценки (10 баллов)

№	Критерий	Баллы
1	ООП и SOLID: качество модели, обоснованность абстракций	4
2	README	2
3	DI-контейнер и подмена зависимости техосмотра в тестах	1
4	Юнит-тесты, в том числе и поведения; ориентир покрытия тестами $\geq 60\%$	2
5	Демонстрация сценария работы приложения и понятный ввод/вывод	0.5
6	Code style	0.5

Штрафы и защита

- до –2 баллов за ошибки во время выполнения;
- до –5 баллов, если программа не собирается;
- снижение пропорционально невыполненным функциональным требованиям;
- до –1,5 балла за не обоснованный overengineering и/или «декоративный» SOLID;